6==========

Scalable Non-Blocking DRAM Memory Subsystem

Adrian Buczkowski
VIP 47922
abuczko@purdue.edu

Heng-I Chu
VIP 47922
chu244@purdue.edu

Eddie Hu
VIP 37920
hu927@purdue.edu

Shams Hoque
VIP 47920
hoques@purdue.edu

Jason Lyst
VIP 37900
jlyst@purdue.edu

Xinyu Liu
VIP 37920
liu3680@purdue.edu

Akram Mohamed
VIP 47920
mahmoud6@purdue.edu

Spring 2026

The feedback for the report is to be returned to:

Adrian Buczkowski
abuczko@purdue.edu

This project was mentored by the following Teaching Assistants:

Akshath Ravikiran, araviki@purdue.edu
Sooraj Chetput, schetput@purdue.edu

Scalable Non-Blocking DRAM Memory Subsystem

Adrian Buczkowski
VIP 47922
abuczko@purdue.edu

Heng-I Chu
VIP 47922
chu244@purdue.edu

Eddie Hu
VIP 37920
hu927@purdue.edu

Shams Hoque
VIP 47920
hoques@purdue.edu

Jason Lyst
VIP 37900
jlyst@purdue.edu

Xinyu Liu
VIP 37920
liu3680@purdue.edu

Akram Mohamed
VIP 47920
mahmoud6@purdue.edu

Abstract

With the growing demand for AI accelerator chips with increasingly large workloads, the large quantity of weights and inputs necessary must constantly be retrieved and stored. To meet these demands, off-chip DRAM (Dynamic Random Access Memory) is required. DRAM, which has a memory capacity on the order of gigabytes, is able to meet the demands in terms of size, but is slow to access. Therefore, to maximize throughput, a memory controller unit and AXI-bus is used to arbitrate the transition of data on and off chip. Previously, our team worked on a blocking controller, which handles requests sequentially, blocking any requests after. To improve upon the design, we are implementing a non-blocking controller, which would allow multiple requests in flight to be processed in parallel. This requires a non-blocking bus; to address this, we propose a lightweight AXI transaction bus that separates Read and Write transactions. The functionality and timing behavior of this design will be confirmed by using a multi-standard DRAM simulation model supporting DDR4, HBM3, and other configurations, integrated through a cycle-synchronized C and SystemVerilog wrapper with a built-in memory controller. This enables flexible, cycle-accurate verification of the full memory subsystem, from on-chip caches and scratchpads through the AXI bus down to DRAM timing and scheduling, across variable standards and configurations without requiring a new controller implementation for each. Testing would occur through extensive simulated transactions as well, as the overlapping nature of transactions requires a large quantity of combinations to be verified. Together, the non-blocking design should perform better in terms of throughput compared to the blocking design, and should match the behavior shown by Ramulator's memory controller, with future plans to leverage this framework for broader DRAM standard exploration beyond DDR4.

1 Introduction and Background

The growing complexity of AI-accelerator chips has resulted in increasingly larger workloads, necessitating the need for a more efficient data storage and access methodology. The magnitude of the data and the subsequent cost of on-chip data storage required the use of off-chip memory. Modern systems rely almost entirely on DRAM as a solution. The SoCET AI Hardware team's custom AI Accelerator Atalla also has the same memory problem, so DRAM was incorporated into the design, with a memory controller to arbitrate requests. More specifically, a memory technology such as DDR (Double Data Rate) and HBM (High Bandwidth Memory) would be used to account for these growing workloads. These technologies can hold large working sets of data while providing more bandwidth, taking advantage of locality, and improving the memory bottleneck.

Previous work on the implementation of the memory controller was based on the flow of memory requests from the chip and the physical structure of DRAM, which is discussed in the previous report [1]. This resulted in a blocking design focused on accuracy as well as providing a realistic representation of DRAM memory. The model was designed to handle requests sequentially and only accept new requests once the current request was completed.

Although the model proved to be functionally correct, it was not efficiently scalable to larger workloads. To improve upon the blocking design to take advantage of DDR4's nature, a non-blocking design was selected as the next iteration of the memory subsystem, following similar designs [2, 3]. The implementation of a non-blocking controller would allow for multiple requests to be in flight, resulting in more bandwidth. Because of the massive amounts of data needed by AI accelerators, largely memory-bound systems are detrimental, making this design choice crucial. However, this would also require a separate communication bus, as the non-blocking nature of the memory controller would possibly result in concurrent load and store requests to occur. Therefore, a split transaction AXI bus was chosen to handle both types of memory access concurrently.

The customized AXI interconnect is responsible for connecting on-chip modules (masters) to the off-chip memory controller (slave). The masters include Scratchpad 0 (SP0), Scratchpad 1 (SP1), Data Cache (D$), and Instruction Cache (I$), each of which issues memory read or write requests []. The main architectural improvement over the blocking interconnect from last semester is the introduction of split transactions and per-channel request buffering. According to the ARM AMBA AXI specification [4], the bus consists of five independent channels: Read Address (AR), Read Data (R),

Write Address (AW), Write Data (W), and Write Response (B). This separation of channels enables split transaction operation, in which read and write transactions proceed independently rather than being funneled into a single unified channel. Each transaction request from the masters is decoupled from its response, allowing multiple transactions to be in flight simultaneously. This means masters are no longer stalled waiting for a response after every transaction, as was the case last semester [1]. This feature allows requests to be buffered within the AXI interconnect, so that DRAM access latency is absorbed mostly by the interconnect rather than by the masters themselves.

Ramulator2 by CMU-SAFARI is a cycle-accurate DRAM timing model that simulates the behavior of modern memory systems such as DDR4 and HBM [5, 6]. Ramulator is capable of modeling the internal organization and timing constraints, such as row activation, precharge, and command scheduling of various DRAM standards, instead of treating memory as a fixed-latency module [6]. Integrating Ramulator into our design allows more flexible evaluation of realistic latency and bandwidth under different accessing patterns, and also enables easy switching between different DRAM standards and configurations without modifying the hardware design. This is especially useful when we want to study the interaction and behavior between our main processor and different DRAM systems under various configurations. In this project, Ramulator is used as the backend memory model to provide a cycle-accurate representation of off-chip memory. In order to integrate the software-based Ramulator with our hardware simulation environment, we deploy DPI-C (Direct Programming Interface for C) to bridge between SystemVerilog and C. DPI-C allows the SystemVerilog modules to directly call C functions and exchange data with Ramulator's frontend interface in real time. This interface establishes communication between the RTL components of our processor and the software DRAM simulator. As a result, this setup creates a co-simulation framework that preserves both hardware-level control flow and software-level memory modeling.

2 Design

2.1 AXI Interconnect

Top-level Idea: The AXI read channel (AR, R) has four masters: SP0, SP1, I$, and D$, as shown in fig.1. The AXI write channel has three masters: SP0, SP1, and D$ [4]. Each path is built with three main components: a per-master manager that absorbs incoming requests into local FIFOs, an arbiter that selects exactly one master per cycle, and a router that directs the subordinate's response back to the originating master by reading the embedded master ID in the data structure. The data width in each path is 64 bits, 8 bytes. This design also entails three key premises: ID assignment, write locking, and maximum outstanding transactions [1]. The block diagrams of the modules described are shown in fig.2.

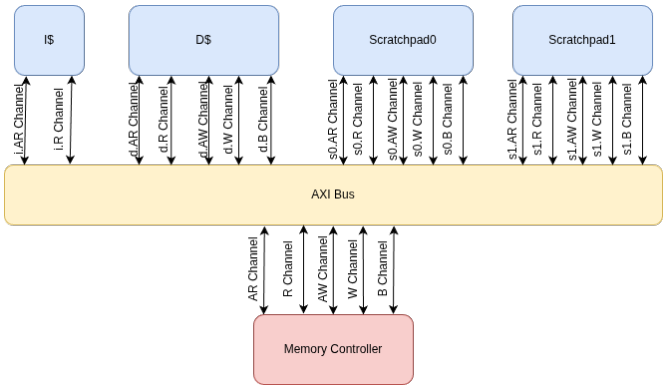

Figure 1: Top-Level Block Diagram of the Customized AXI Interconnect

ID Assignment: To achieve out-of-order transactions, two IDs are generated: one from the master and one from the AXI interconnect. The master-generated ID is used by each master to match requests sent out with responses received from the interconnect. The AXI-generated ID is used by the interconnect to match each request and response to the originating master. Both IDs from each request must be echoed back in the slave's response to fulfill this purpose [1].

Locking Write: AXI3 allows the W and AW channels to be fully decoupled and interleaved [4]. To simplify the design for both the interconnect and the DRAM controller (the slave), the W and AW channels are coupled and operate in lock-step. This design choice sacrifices some bandwidth.

Max Out-standing Transaction: Because masters are not stalled after they send out requests [4], the maximum number of outstanding transactions must be limited. Each master is allowed up to 4 outstanding requests.

Read/Write Manager: The manager is responsible for Valid/Ready handshaking with the master and buffering outstanding requests [1]. Each read manager has a FIFO to store requests, which are popped upon receiving a grant from the arbiter. Each write manager has both a request FIFO and a data FIFO, since both the request and data must be buffered. The data FIFO has a capacity of 8 beats per request, for a total of 32 beats of data capacity.

Read/Write Arbiter: The arbiter is responsible for issuing grant signals to managers who are waiting to send requests [1]. The priority order from high to low is: SP0, SP1, D$, and I$. When a master has multiple pending requests, scheduling follows the round-robin policy. When no requests are present, the arbiter enters an IDLE state.

Read/Write Router: The router is responsible for routing responses from the slave back to the correct master, by reading the master ID embedded in the response [1].

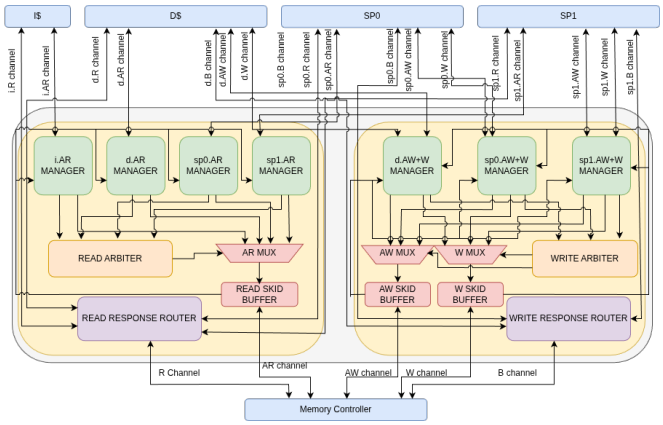

Figure 2: Top-Level Block Diagram of the Customized AXI Interconnect

2.2 Memory Controller

The Non-blocking Memory Controller is directly connected to the split-transaction AXI bus and is made up of a DRAM command path with data transfer modules for reading and writing data to the DRAM. Data transfer is synchronized with the DRAM command path, allowing data to travel directly from the DRAM to an intermediary queues, corresponding to each path, before being transferred to the AXI bus or DRAM. The general connections are shown in fig. 3.

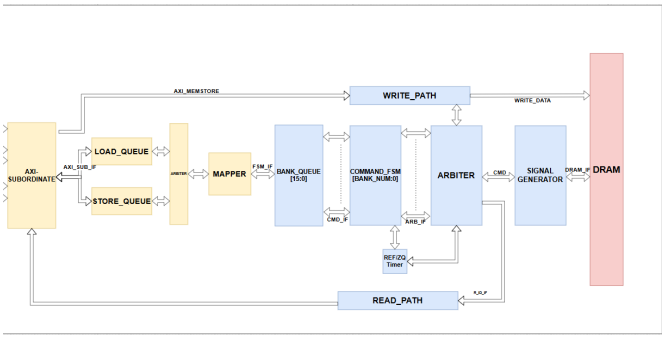

Figure 3: Top-Level Diagram of Non-blocking Memory Controller

2.2.1 DRAM Command Path Frontend

The DRAM command path consists of queues and arbitrators designed to provide read and write commands to the DRAM to maximize performance by taking advantage of the open row policy.

Requests that flow through this path carry the physical memory address, the id, and the length of the request in ”beats” within the burst that contains meaningful information. This information is sent to the store and load queues, which are circular FIFOs, depending on the type of request that is given when it is valid. The Frontend Arbiter then looks within these queues, using round-robin priority to grant these queue slots into its own. Once it receives a request, the Frontend Arbiter uses an address mapper to convert the AXI address into a bank, bank group, column, and row. Based on these values, the request will then be sent to the bank queues.

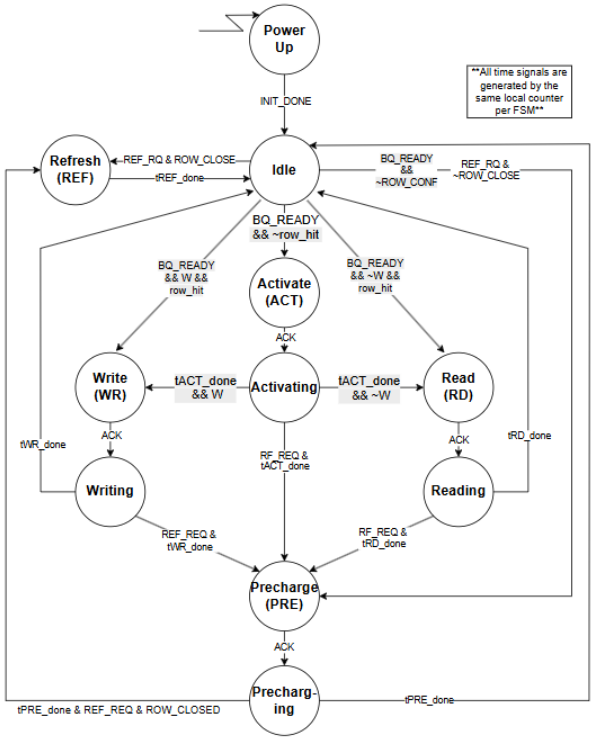

Figure 4: Command FSM State Diagram

2.2.2 DRAM Command Path Backend

Once the request passes through the Frontend Arbiter, it is considered to be in the backend. The requests then enter the bank queue module, which contains a number of FIFO queues equal to the number of DRAM banks that act in parallel, each belonging to a specific bank. These banks then trigger the corresponding command FSM, whose states depend on the DDR4 timing parameters to transition, which is discussed more in the previous report [1] as well as in the JEDEC guide for DDR4 and Micron Technology [7, 8]. Each bank has its own command FSM, so the timings of each command occurring in a bank will not be seen by the other banks (bank-level parallelism). The Command FSM state diagram can be found in fig. 4.

The Command FSMs are connected directly to the Backend Arbiter, which handles the final transfer. It acts as a one-hot rolling shift register module, constantly checking the status of both the command FSMs and the DRAM banks. These are the key optimizations that allow the controller to be non-blocking. By having bank-level parallelism, many commands can be sent to the DRAM from each bank. Delays from one bank are not propagated to other banks, so requests are not blocked among banks. Only commands within a bank cannot be sent immediately after each other due to their internal timings. When the Backend Arbiter selects a specific command FSM, it is granting it permission to use the DDR4 timing parameters to

transition to its next state. Using these timing parameters, the Backend Arbiter waits until an FSM is ready to read or write before passing the address of the command as well as signaling for the write/read path to send the accompanying data to the target, such as from the Write Data Path to the DRAM or from the DRAM to the Read Data Path.

2.2.3 Read and Write Paths

The Read and Write Paths are a series of queues in parallel that are connected to the AXI, the Backend Arbiter, and the DRAM physical interface. Requests are separated according to their target ID, and their corresponding data being loaded into the respective queue. Once the bank is ready to be accessed, the transfer is then signaled by the Backend Arbiter.

During write operation, the AXI bus stores the data of write requests within the write path's queue, and waits until the queue is signaled by the backend arbiter, who notifies the queue when the destination bank is able to be written to. For read operations, the flow is reversed, with the structure remaining largely similar outside of a second set of parallel queues for the ID of the read data. Data transfers still wait on the signal from the Backend Arbiter, which directs the read data to the respective queue as well as loading the ID queue. Both groups of data are then transferred to the AXI bus and back onto the chip.

2.3 Ramulator Integration

The Ramulator integration consists of two closely communicating layers, the SystemVerilog AXI subordinate wrapper and the C Ramulator wrapper. These two layers jointly create an interface between a standard AXI4 subordinate [4] and the memory subsystem while accurately modeling the DRAM timing and scheduling behavior defined by Ramulator [5, 6]. Specifically, the SystemVerilog wrapper connects directly to the AXI channels and uses a 256 bit data interface between the AXI bus and the wrapper. It handles request queuing, reordering, and flow control in synthesizable RTL, and communicates with the C layer through DPI-C imported functions. The C wrapper instantiates Ramulator's frontend and memory system, and maintains a functional memory model during simulation.

The integration is highly configurable through external YAMl files, allowing the same wrapper to support multiple DRAM standars such as DDR4, HBM2, and HBM3 without modyfing the RTL. Key parameters including organization (channels, banks, rows), timing presets, address mapping schemes, and scheduling policies are defined in the configuration file. This enables rapid design-space exploration. where different memory architecture and policies (e.g., FR-FCFS scheduling, open-row vs. closed-row policies) can be evaluated under the same AXI traffic conditions. As a result, the platform serves not only as a functional memory model but also as a performance analysis tool.

The address mapping between AXI requests and DRAM organization is handled within the Ramulator configuration using schemes such as RoBaRaCoCh and ChRaBaRoCo. These mappings determine how physical addresses are distributed across channels, ranks, banks, rows, and columns, directly impacting row buffer locality and bank-level parallelism. Choosing an appropriate mapping is critical for performance, as it influences row hit rates and reduces access conflicts across banks.

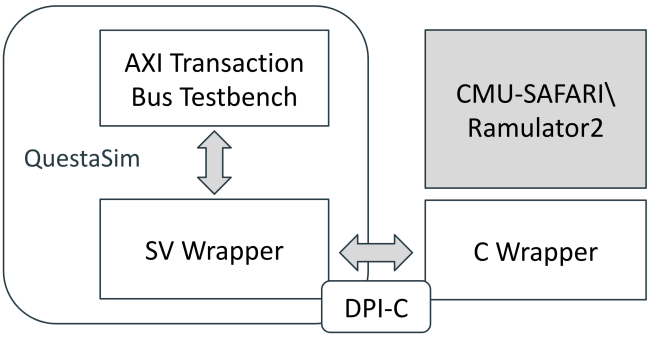

Figure 5: AXI–Ramulator co-simulation framework using DPI-C interface

In addition, the C wrapper handles memory interactions such as request submission and response collection, while accurately representing the timing behavior from Ramulator at the same time [5]. Since our simulation runs at 800 MHz while DRAM standards such as DDR4 run at 1600 MHz, the clock mismatch between the AXI bus and Ramulator's internal clock is handled by issuing a predefined number of Ramulator ticks per AXI cycle. This allows the same wrapper to operate correctly across different DRAM standards without modifying the RTL design. As a result, this setup provides a flexible co-simulation platform for evaluating the full memory subsystem.

2.3.1 Read Path

The read path of the AXI-to-Ramulator wrapper handles both single-beat reads, where the burst length field equals zero, and burst reads, where the burst length is greater than zero. Both types of requests share the same AXI read address channel, but differ in how they are accepted and tracked [4]. Single-beat reads issue one memory request to Ramulator with a sentinel source ID, and the corresponding AXI ID is recorded in a lookup table accessed by address. After completion from Ramulator, the 256 bit response is assembled from four 64 bit words with their corresponding IDs and written into a single-beat response FIFO of depth 8192. Requests will only be accepted if the response FIFO has available space and the memory system accepts the request, otherwise the read address channel will be stalled.

Burst reads are treated differently from single-beat reads as they require extra reordering. This is because AXI enforces in-order responses within a transaction [4], yet the memory system may return responses out of order. To tackle this problem, the wrapper maintains an 8192 slot reorder buffer, where each entry stores per-beat data, valid bits, and transaction metadata. After a request is accepted, a free slot will be allocated in their

original order, and all beats will be issued to the memory system identified by their first index. When the response is returned, it will be routed back to the correct slot and placed using their addresses as beat indices. At the output stage, in order to maintain correct ordering, only the oldest active burst will be able to drive the read data channel and return beats sequentially. This process will be stalled if the next beat is not ready. Once all beats are completed, the slot will be released back to the free list. This design not only supports incrementing, wrapping, and fixed burst types [4], but also prioritizes burst responses over single-beat responses to avoid interleaving.

during simulation, several performance bottlenecks were observed at the interface between the AXI wrapper and the memory model. In particular, backpressure happens when the memory system can't accept new requests, which propagates to the AXI channels by deasserting ready signals. Additionally, burst transactions introduce stalls when reorder buffer entries are not fully populated, preventing in-order response delivery. These effects highlight the importance of balancing queue sizes, reorder buffer depth, and memory scheduling policies to maintain high throughput.

2.3.2 Write Path

The write path ensures that only one write transaction can be processed at a time using the active flag, so a new write address will not be accepted until the current transaction has completed. For single-beat writes, a transaction is accepted as long as there is available space in the write response FIFO. The 256 bit data beat is divided into four 64 bit sub-beats and issued independently as memory write requests. For burst writes, a transaction is accepted only when the address and the first data beat arrive in the same cycle and the write response FIFO has available space. At that point, the address metadata is latched and the channel remains active for the remaining beats. Each 256 bit data beat is then processed in the same way, with sub-beat addresses derived based on the burst type [4] and a beat counter tracking the remaining number of burst beats.

Byte strobes are applied after dividing the 256 bit data into four 64 bit sub-beats. When a partial write is triggered, unlike where all bytes are enabled and data is written directly, a read to the memory model will be performed to extract the existing data for the masked bytes. Then, the old and new data will be merged according to the byte mask [4], and the write operation will be processed as usual. All sub-beat requests must be accepted by the memory system in the same cycle, or otherwise the write data channel will be stalled. When the final beat of the transaction is issued, a write response will be enqueued into the FIFO and the active flag will be cleared. The response channel then drains the FIFO after receiving a handshake from the master [4], and both the address and data ready signals will be asserted.

2.3.3 Control Flow

The control flow and arbitration logic of the SystemVerilog wrapper is designed using a seven-stage pipeline to keep all operations ordered within a single clock cycle. At the beginning of each cycle, mutable components such as the FIFO counters, pointers, and free slot tracking list are copied into blocking shadow variables. These variables record all updates within the same cycle and are committed at the end of each cycle. All data flow and decisions are made based on the shadow variables so that every stage shows a consistent status of the entire system.

On the read side, the address ready signal is used to show the availability of resources. It is deasserted if the reorder buffer is full or when the single-beat response FIFO is full, and it is asserted only when a request can be accepted and issued to the memory system. In particular, single-beat requests are not accepted if the FIFO is full or if the memory system does not accept the request. In contrast, burst requests are not accepted if the reorder buffer slots are full or if the first beat cannot be issued. On the write side, the address and data ready signals are cleared at the beginning of each cycle and are only asserted for one cycle per handshake to prevent repeated acceptance [4]. Write transactions are also handled using an active flag so that a new write address is not accepted until the current transaction finishes. The flag is set when the first write beat is accepted and is cleared when the final beat is processed.

In order to stop transactions from issuing before the memory model is ready, all request processing is gated by an initialization signal. The read data channel uses a register to hold a response until it is accepted by the master. Burst responses are prioritized over single-beat responses to avoid interleaving and to preserve ordering within a burst [4]. At the end of each cycle, all updates stored in the shadow variables are committed to the registers in one step to ensure that the next cycle starts from a consistent status.

3 Results

3.1 AXI Interconnect

A SystemVerilog testbench was created to ensure the correctness of the design [9]. The testbench utilizes a UVM-style environment, including input drivers, monitors, and a scoreboard to validate data integrity.

The bandwidth (BW) of both write and read channels was measured at zero latency, meaning responses were provided by the slave instantly upon request. The bandwidth is defined and calculated using the following formula (1):

$$BW = \frac{N_{beats} \times W_{bytes}}{T_{cycles}} \tag{1}$$

Where:

- N_{beats} is the total number of data beats transferred.
- W_{bytes} is the data bus width in bytes (8 B for this 64-bit design).

- T_{cycles} is the total number of clock cycles elapsed during the measurement window.

The results from the simulation stress tests with randomized burst length, for both blocking and customized AXI interconnect, are summarized in Table 1. The data highlights the performance gap between the read and write paths, specifically noting the lower utilization of the write channel due to the lock-step nature of the AW and W channels in the current implementation.

3.2 Memory Controller

In order to verify the functionality of the non-blocking memory controller, one main metric was used: bandwidth. The utilization is found by dividing the time to transact 512 bits of data by the time to complete a memory transfer, and throughput is determined by the utilization and peak bandwidth. Equations for these metrics can be found in Equation 2.

$$U^{\text{blk}} = \frac{t_{\text{BURST}}}{\Delta}, \qquad \text{Throughput} = U^{\text{blk}} \cdot R_{\text{peak}} \tag{2}$$

To accurately simulate the controller, Micron models and test-benches were used to replicate the realistic operation of the DDR4 DRAM that would be used for off-chip memory [10]. To model the results, a best-case, worst-case, and average case metric was used. The average case used 10K random cases, best case used 10K row hit cases, and the worst case used 10K row miss cases.

The results from the simulation is consistent with what is expected for a non-blocking DRAM controller. The controller uses significantly lower time and higher bandwidth than the blocking controller. Due to overlapping requests, more bandwidth can be used, thoroughly increasing the performance of the memory controller. This will allow the Atalla processor to be less memory-bound in its computations. The results can be found in Table 2.

3.3 Ramulator Integration

The correctness of the memory subsystem is validated before conducting the performance evaluation. The validation is performed by directly simulating AXI transactions through the wrapper rather than running the full processor system. Specifically, all read data returned from the AXI interface is compared against a reference memory model using a shadow memory, and we have observed no mismatches across all test cases. In addition, we perform a comprehensive set of stress tests such as read-after-write, raw reads, read and write backpressure, simultaneous read and write requests, and multiple outstanding transactions to validate the non-blocking behavior. We also verify different burst types such as INCR, FIXED, and WRAP [4], as well as write strobe byte masking [4] to ensure correctness under various accessing patterns.

The performance benchmark is conducted by simulating AXI transactions that emulate an SDMA (Streaming Direct Memory Access) instruction, which loads a 1024×1024 int16 matrix from memory through the AXI interface. All measurements and timing parameters are taken in SystemVerilog simulation cycles at 800 MHz. In this benchmark, we compare DDR4_8Gb_x16 using the DDR4_3200w timing preset and RoBaRaCoCh (Row, Bank, Rank, Column, Channel) address mapping across configurations of 1, 2, and 4 channels with 128, 256, and 512-bit interfaces per channel, as well as HBM3_8Gb using the HBM3_2Gbps timing preset and MOP4CLXOR (multi-channel, column-level XOR-based mapping) address mapping across 1, 8, and 16 channels, each with 2 pseudo-channels and 32-bit interfaces. All configurations are implemented with an open-row policy and an FR-FCFS (First ready, First come, First serve) scheduler, along with 256-bit read and 64-bit write buffers in the memory controller. These configurations are designed to maximize performance by reducing row misses and bank conflicts while exploiting spatial locality.

The benchmark evaluates row-major (1024-element rows) and tile-major (32×32-element tiles) accessing patterns. Table 3 shows the single-beat mode results, and Table 4 shows the burst mode results across different memory types, channel counts, and interface widths. Each DRAM configuration evaluates both accessing patterns for more intuitive comparison. The Cycles column shows the total simulation cycles to complete the workload, and Time (μs) shows the real execution time by converting clock cycles using an 800 MHz clock. Cyc/row shows the average cycles per matrix row, Cyc/8-beat shows the average cost of an 8-beat transfer, and Cyc/inter shows the average delay between two consecutive data beats. B/cycle shows the effective throughput in bytes per cycle, and Stall shows the percentage of cycles where new requests are blocked due to backpressure or resource contention.

4 Evaluation and Future Work

4.1 AXI Interconnect

After thorough verification and stress testing [9], we conclude that the customized AXI interconnect achieves both correctness and reliability when transmitting large amounts of data. This semester's work provides meaningful insight that non-blocking transactions increase bandwidth, and it is worthwhile to further optimize the design.

Although the customized AXI interconnect showed an increase in bandwidth compared to the blocking interconnect from last semester, we still cannot reach the theoretical limit provided by the Ramulator. The bottleneck sits at the data width of each channel, with only 8 B/cycle at full utilization, whereas Ramulator provides up to 30 B/cycle max bandwidth from DRAM. Apart from the theoretical limit, the customized AXI interconnect failed to reach full utilization of 8 B/cycle, and the bottleneck mainly lies within the write channel. With lock-step operation, the write channel's bandwidth is limited due to less flexibility for the master when storing data.

To address these two main bottlenecks, future developers can try to improve the design from the following angles:

Table 1: Performance Comparison: Previous Blocking Controller vs. Current Non-blocking AXI Interconnect

Operation	Blocking Interconnect [9]		Customized AXI Interconnect		
	Bandwidth	Utilization	Cycles	Bandwidth	Utilization
Read Path	0.820 B/cycle	10.25%	10,010	3.280 B/cycle	41.00%
Write Path	0.890 B/cycle	11.13%	8,000	2.225 B/cycle	27.81%

Case	Total Time (ns)	Utilization (%)	Bandwidth (GB/s)
Best (B)	8115563	7.39	0.39
Average (B)	24199105	2.48	0.13
Worst (B)	30771793	1.95	0.10
Best (NB)	272853	-	1.16
Average (NB)	3054255	-	1.03
Worst (NB)	7692948	-	0.40

Table 2: Non-Blocking VS Blocking Memory Controller Results

First, increasing the number of transaction channels. Multiple instances of the current custom AXI interconnect can be implemented in parallel. We can keep the data width for each channel but increase the total burst data width up to 128 bits by using two AXI interconnects in parallel.

Second, increasing the data width of each channel. Without implementing two separate AXI interconnects, which would double area and power consumption, the data width in each channel can be increased from 64 bits to 128 bits, which naturally doubles the current bandwidth.

Third, enabling write interleaving [4]. Similarly to standard AXI3, where the first beat of write data is in order but all remaining beats can be interleaved, the W channel would not be lock-stepped with the AW channel, which will theoretically increase the bandwidth.

4.2 Memory Controller

The results show that the non-blocking controller performs better than the blocking version. The bandwidth is nearly eight times higher than that of the blocking controller in the average case. The non-blocking controller is able to send commands to DRAM while others are in flight, due to the bank level parallelism that comes from the bank queues. This will allow the Atalla processor to be less memory bound in its computations.

For future work, several considerations have been made. One such optimization is the reordering of requests within banks, allowing for younger instructions to be executed before older ones. This would be done to maximize row hits and take advantage of the open row policy. This would lead to higher throughput, but could add to area due to added hardware. Another consideration is to customize the data widths that are returned in a burst. In the implemented controller, only 64 bits of data are transferred per burst. To accommodate for this without a PHY, several changes to the DRAM interface and state machines would be needed. This leads to another optimization: integrating the controller with a physical PHY. Currently, the controller is being verified with a DDR4 Micron simulation model. By using a real PHY, the controller can be taped out later on by revealing the analog and mixed signal nonlinearities that cannot be seen using a simulation model. Finally, the controller can be made for HBM. Currently, the controller is being made for DDR4, but with more advanced hardware and logic, the controller can be customized to fit HBM's structure.

4.3 Ramulator Integration

We have observed that the accessing pattern only matters when the system does not have enough bandwidth. Specifically, in the single-beat results, tile-major access tends to perform better than row-major in low-bandwidth DDR4 test cases. In DDR4 1 channel 128 bits, throughput increases from 10.90 to 15.18 B/cycle, and stall decreases from 68.0% to 52.2%. This result demonstrates that tile-major access can better exploit spatial locality and reduce row conflicts when resources are limited. However, when we increase the bandwidth, we observe the opposite behavior. In DDR4 4 channels 128 bits and all HBM3 cases, both accessing patterns produce almost identical results, showing that the access pattern no longer serves as the bottleneck.

Furthermore, as we increase the number of channels, we observe the most improvement in performance. Specifically, from DDR4 1 channel to 2 channels, we see almost 2 times speedup and a significantly lower stall percentage, because each channel can operate independently with its own banks, allowing requests to be distributed instead of competing for the same resources. In DDR4 4 channels, excessive stalls are eliminated in single-beat mode and throughput reaches around 32 B/cycle, approaching the theoretical ceiling, since the controller is able to keep multiple channels active at the same time and maintain a steady flow of requests. We can conclude that having more channels increases parallelism and allows more requests to be processed at the same time. On the other hand, simply increasing interface width does not show as

Table 3: Memory Access Performance (Single-beat Mode)

Config	Pattern	Cycles	Time (μs)	Cyc/row	Cyc/8-beat	Cyc/inter	B/cycle	Stall (%)
DDR4 1ch 128b	Row	207824	259.78	202	3	3	10.90	68.0
	Tile	138126	172.65	134	2	2	15.18	52.2
DDR4 1ch 256b	Row	242003	302.50	236	3	3	8.66	72.5
	Tile	190603	238.25	186	2	2	11.00	65.2
DDR4 1ch 512b	Row	259211	324.10	253	3	3	8.90	74.3
	Tile	259510	324.38	253	3	3	8.80	74.2
DDR4 2ch 128b	Row	104145	130.18	101	1	1	20.13	36.1
	Tile	90679	113.34	88	1	1	23.12	26.9
DDR4 4ch 128b	Row	65924	82.40	64	1	1	31.81	0.0
	Tile	66299	82.87	64	1	1	31.63	0.0
HBM3 1ch 2pc 32b	Row	65558	81.94	64	1	1	31.98	0.0
	Tile	65604	82.00	64	1	1	31.96	0.0
HBM3 8ch 2pc 32b	Row	65556	81.94	64	1	1	31.99	0.0
	Tile	65561	81.95	64	1	1	31.98	0.0
HBM3 16ch 2pc 32b	Row	65556	81.94	64	1	1	31.99	0.0
	Tile	65566	81.95	64	1	1	31.98	0.0

much improvement. For example, DDR4 1 channel 256 bits and 512 bits perform worse than 128 bits, with higher stall and lower throughput, because a wider interface only increases the amount of data per transfer but does not increase the number of concurrent memory operations. Therefore, without enough parallel requests to utilize the available bandwidth, the wider interface cannot be fully exploited.

Moreover, burst mode greatly boosts performance when bandwidth is limited. Specifically, in DDR4 1 channel 128 bits, we observe a jump from 10.90 to 29.61 B/cycle, which is about 3 times speedup when comparing single-beat mode with burst mode. We believe this is because burst transfers amortize overhead for each request and allow multiple transaction beats to be processed under a single command, maximizing the use of available bandwidth. However, for high-bandwidth configurations such as DDR4 4 channels 128 bits and HBM3, burst mode does not show a significant improvement. We believe this is because these systems are already close to their maximum throughput, and the memory pipelines are already well utilized.

Finally, we observe strong performance with HBM3 across all test cases. Specifically, even with only 1 channel, it achieves about 32 B/cycle with 0% stall in single-beat mode, which is similar to DDR4 4 channels 128 bits and close to the theoretical ceiling of DDR4 standards. We believe this is because its architecture provides strong bank-level parallelism and pseudo-channel organization, allowing multiple requests to be served efficiently and concurrently. Therefore, increasing the number of HBM3 channels does not further reduce cycles, which also means the workload is no longer limited by the memory system, and the system has already reached its throughput limit. This result demonstrates the potential of continued research in integrating HBM3 standards with more optimized pipelines and scheduling. We foresee a positive trend in performance if improvements in request scheduling, address mapping, and pseudo-channel utilization are implemented to maintain high efficiency under more complex and irregular workloads.

There are several limitations in our current design and evaluation. First, the HBM3 wrapper is not fully implemented. We only verified basic correctness but not optimizations such as deeper buffering, better scheduling, and improved handling logic. Second, the AXI interconnect and the Ramulator wrapper are not fully integrated with the complete processor system. We were only able to evaluation based on simulating AXI transaction, which does not represent the realistic workload behavior. Third, the AXI interconnect itself introduces significant overhead, and the overall bandwidth utilization is still low due to the relatively slow AXI bus and serialized write path. Forth, the fixed buffer sizes within the wrappers may become bottlenecks if we continued to explore more complex access patterns. Also, although Ramulator is able to provide cycle-accurate timing, it does not reflect lower level physical effects like PHY behavior or power. For future work, the HBM3 wrapper can be further optimized to better utilize pseudo-channels and parallelism. The AXI interconnect can be improved to reduce overhead, and we should fully integrate the entire processor system explore more realistic benchmarking. In addition, we can also explore more advanced scheduling policies, address mappings, and adaptive buffering to further improve performance.

5 Conclusion

As seen in our findings, it is clear that the implementation of high-bandwidth memory subsystems to interface with DRAM is critical to the high-performance operation of AI-accelerator chips. As workloads continue to grow in size and complexity, the demand for memory bandwidth also increases, making memory the primary bottleneck. The results show that systems with higher memory-level parallelism, especially those

Table 4: Memory Access Performance (Burst Mode)

Config	Pattern	Cycles	Time (μs)	Cyc/row	Cyc/8-beat	Cyc/inter	B/cycle	Stall (%)
DDR4 1ch 128b	Row	70802	88.50	69	1	1	29.61	51.9
	Tile	69578	86.97	67	1	1	30.14	51.4
DDR4 1ch 256b	Row	104017	130.20	101	1	1	20.16	67.5
	Tile	87037	108.79	84	1	1	24.90	61.4
DDR4 1ch 512b	Row	121273	151.59	118	1	1	17.29	72.1
	Tile	121262	151.57	118	1	1	17.29	72.1
DDR4 2ch 128b	Row	65737	82.17	64	1	1	31.90	25.2
	Tile	65647	82.50	64	1	1	31.94	25.1
DDR4 4ch 128b	Row	65562	81.95	64	1	1	31.98	25.0
	Tile	65686	82.10	64	1	1	31.92	25.2
HBM3 1ch 2pc 32b	Row	65552	81.94	64	1	1	31.99	25.0
	Tile	65588	81.98	64	1	1	31.97	27.9
HBM3 8ch 2pc 32b	Row	65552	81.94	64	1	1	31.99	25.0
	Tile	65564	81.95	64	1	1	31.98	25.0
HBM3 16ch 2pc 32b	Row	65552	81.94	64	1	1	31.99	25.0
	Tile	65572	81.96	64	1	1	31.98	25.0

using HBM, are able to provide much higher throughput, while traditional DDR4 configurations require significant changes to reach similar performance. Therefore, we propose that future systems be designed towards HBM integration, as reflected by the performance improvements observed in the simulations. At the same time, Ramulator will serve as the main research module due to its flexibility in modeling different memory standards, configurations, and timing behaviors, allowing the exploration of design tradeoffs. The non-blocking memory controller and split-transaction AXI bus developed in this paper will serve as a tested solution, providing a practical and scalable hardware implementation that can efficiently utilize the available memory bandwidth and can be further improved in future work.

References

[1] A. Buczkowski, A. Kadakia, D. Khatri, J. Lyst, and T. Than, "DRAM memory controller," 2025, Purdue University, Fall 2025 Blocking Controller Report.

[2] D. Germchi, "A high performance DDR4 memory controller on FPGA," Master of Applied Science Thesis, University of Waterloo, Waterloo, Ontario, Canada, 2024, accessed: Jan. 16th, 2026. [Online]. Available: https://dspacemainprd01.lib.uwaterloo.ca/server/api/core/bitstreams/510feb76-4250-457c-966e-ce8fbbc1242a/content

[3] B. Jacob, S. Ng, and D. T. Wang, *Memory systems: cache, DRAM, disk*. Burlington, MA: Morgan Kaufmann Publishers, 2010.

[4] *AMBA AXI and ACE Protocol Specification*, ARM Limited, 2021, iHI 0022H.c.

[5] C. SAFARI Research Group. (2023) Ramulator 2.0: A modern, extensible dram simulator. Accessed: Apr. 2026. [Online]. Available: https://github.com/CMU-SAFARI/ramulator2

[6] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," in *IEEE Computer Architecture Letters*, 2015, accessed: Apr. 2026.

[7] *JEDEC Standard JESD79-4: DDR4 SDRAM*, JEDEC Solid State Technology Association, September 2012, accessed: Dec. 11, 2025. [Online]. Available: https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/196/JESD79_2D00_4.pdf

[8] *Micron 8Gb DDR4 SDRAM Datasheet*, Micron Technology, 2025, accessed: Dec. 12, 2025. [Online]. Available: https://www.mouser.com/datasheet/2/671/Micron_8gb_ddr4_sdram-3239981.pdf

[9] Purdue SoCET Team. (2026) Atalla axi interconnect testbench (axi_tb.sv). Accessed: Apr. 23, 2026. [Online]. Available: https://github.com/Purdue-SoCET/atalla/blob/axi_bus_main_xinyu/tb/unit/memory/axi_bus/axi_tb.sv

[10] *Simulation Models and Technical Documentation*, Micron Technology, 2025, accessed: Dec. 12, 2025. [Online]. Available: https://www.micron.com/

A Teamwork

The work done was split into three separate teams: The AXI team, the Non-blocking Memory Controller team, and the Ramulator team. The idea was that each team would assign specific sections of the overall team's work to a member, with ample cross-communication to make sure designs were in line

with each other. First, the individual sub-modules were individually designed and verified, with shared input/help from other team members when necessary. Finally, top-level designs were verified together as a team.

The AXI team consisted of Xinyu Liu and Aryan Kadakia. Aryan, a volunteer on the team who worked on the blocking variation of the memory controller, worked on the design of the write path, while Xinyu worked on the design and verification of the read path. Aryan and Xinyu worked on verification of the write path collaboratively.

The non-blocking team had four members, Adrian Buczkowski, Shams Hoque, Eddie Hu, and Jason Lyst. First, the controller was broken up into the front-end and backend, with Adrian and Shams working on the front-end while Eddie and Jason worked on the backend. There were 8 ”true” sub-modules that comprised the memory controller with some being similar enough to have a wrapper of a general module - Read/Write and Store/Load. On the front end, Shams worked on the Load Queue and Store Queue, with Adrian working on the Front End Arbiter and Command FSMs. For the backend, Eddie worked on the Bank Queue, with Jason working on the general Read/Write and Backend Arbiter. Verification remained larger the same, with Eddie working on the Verification of the Backend Arbiter. Then, top-level integration and testing was completed by the whole team through adapting the previous blocking testbench to suit the needs of the new controller.

Finally, the Ramulator team had two members, Heng-I Chu and Akram Mohamed. Heng-I worked on designing for all of the SystemaVerilog wrapper, C wrapper, Ramulator interface, functional model, optimized buffer and control logic, testbench, and AXI transaction emulator. Akram worked on assisting the development of HBM3 support and Ramulator interface.